UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL LA PLATA

INGENIERÍA EN SISTEMAS DE INFORMACIÓN

TRABAJO PRÁCTICO INTEGRADOR N.º 4

GESTIÓN OPERATIVA Y SEGURIDAD EN REDES

ARTÍCULO DE REVISIÓN DEL ESTADO DEL ARTE

Del SNMP a la observabilidad moderna

Estado del arte del monitoreo de redes con Prometheus y Grafana

GRUPO 5

Ian Vidmar · Juan Atencio · Patricio Borda

La Plata, Buenos Aires · Noviembre de 2026

Resumen

[Resumen pendiente: 100–150 palabras.]

Palabras clave: monitoreo de redes; observabilidad; Prometheus; telemetría; AIOps.

1. Introducción

[Introducción pendiente — Patricio: contexto, objetivo y alcance (½ página).]

2. Estado del arte

El monitoreo permite conocer el estado de equipos y servicios; la observabilidad, además, busca explicar sus fallas a partir de diferentes evidencias. La transición es relevante en redes y aplicaciones distribuidas, donde un incidente puede involucrar varias capas [1], [2].

2.1. Instrumentación y monitoreo tradicional: SNMP, Syslog y NetFlow/IPFIX

SNMP permite consultar, desde un sistema gestor, objetos de información definidos en MIB e identificados mediante OID. Los agentes de routers, switches y servidores exponen contadores, errores y estados, y pueden enviar notificaciones. Su amplia adopción facilita gestionar equipos diversos, pero las diferencias entre MIB y los intervalos de sondeo pueden dificultar la detección de fallas breves [1], [2]. SNMPv3 incorpora autenticación y protección de los datos de gestión [3].

Syslog registra y clasifica mensajes de eventos útiles para investigar fallas o cambios de configuración [4]. NetFlow e IPFIX describen flujos mediante direcciones, puertos, protocolos y volúmenes de tráfico; IPFIX estandariza su exportación [5]. Son fuentes complementarias: SNMP observa variables de equipos; Syslog, eventos; e IPFIX, comunicaciones. Ninguna permite por sí sola reconstruir completamente un incidente distribuido.

2.2. Monitoreo basado en métricas: Prometheus, exporters y Grafana

Prometheus recolecta métricas mediante consultas periódicas a endpoints HTTP, las almacena como series temporales identificadas por etiquetas y permite analizarlas con PromQL. Sus reglas pueden activar alertas cuando se cumple una condición. Los exporters adaptan fuentes de datos: node_exporter publica indicadores del entorno Linux y blackbox_exporter comprueba servicios desde el exterior, por ejemplo, mediante HTTP. Este enfoque facilita automatizar la supervisión, aunque requiere controlar el volumen de series y proteger sus endpoints [6].

Grafana consulta fuentes como Prometheus y presenta paneles de disponibilidad, latencia y uso de recursos [7]. Permite visualizar tendencias y relacionar indicadores, aunque un gráfico no demuestra por sí mismo la causa de una falla.

2.3. Telemetría de red: polling y streaming con gNMI

En el polling, el colector consulta los equipos a intervalos fijos: su configuración es simple, pero puede omitir eventos breves. En la telemetría por suscripción, el equipo entrega actualizaciones según las condiciones acordadas, lo que mejora la oportunidad de ciertos datos, aunque exige administrar conexiones y volúmenes de información [2].

gNMI, especificado por OpenConfig, utiliza gRPC y modelos de datos estructurados para consultar estado y configuración y recibir telemetría. Su operación Subscribe

admite ONCE, POLL y STREAM; este último puede enviar muestras periódicas o cambios de estado [8]. Por eso, gNMI no se limita a un esquema de envío continuo. La elección entre sondeo y suscripción depende de la precisión requerida, las capacidades de los equipos y el costo operativo.

2.4. Comparación sintética de enfoques

La Tabla 1 resume fuentes, aportes y limitaciones. Los enfoques no se reemplazan necesariamente: una organización puede combinar SNMP, Syslog, IPFIX y métricas de Prometheus según sus necesidades.

Tabla 1. Comparación de enfoques de monitoreo y telemetría de redes.

Enfoque	Datos y adquisición	Aportes	Limitaciones y seguridad
SNMP / Syslog / IPFIX	Contadores sondeados; eventos; flujos exportados	Equipos heredados; registro de eventos y tráfico	Intervalos de muestreo; MIB heterogéneas; autenticación y privacidad
Prometheus, exporters y Grafana	Scraping de métricas y sondeos de servicios	Series temporales, consultas, alertas y visualización	Cardinalidad; protección de endpoints; no explica causas por sí solo
gNMI	Modelos estructurados; ONCE, POLL y STREAM	Suscripciones y actualizaciones detalladas	Soporte variable; volumen de datos y controles de acceso
OpenTelemetry / eBPF	Instrumentación de señales y eventos del kernel	Correlación y visibilidad adicional de aplicaciones y tráfico	Instrumentación; costes, permisos y seguridad

Fuente: elaboración propia a partir de [1]–[8].

2.5. Caso ilustrativo: demostración de monitoreo y alertas

La demo del Grupo 5 utiliza Docker Compose con Prometheus, Grafana, node_exporter, blackbox_exporter, Nginx y Alertmanager. Prometheus recolecta cada cinco segundos métricas de infraestructura y comprueba la disponibilidad HTTP de Nginx; Grafana muestra estado, duración de sondeos, recursos y alertas. No implementamos SNMP ni gNMI: el caso ilustra monitoreo de servicios e infraestructura.

El ensayo previsto consiste en detener Nginx: blackbox_exporter deberá registrar probe_success = 0 y la regla ServicioCaido se activará si la condición se mantiene 30 segundos. Alertmanager está configurado para enviar el aviso por webhook a un puente MQTT, que intenta reenviarlo a ntfy cuando hay internet. Tras reiniciar el servicio deberá observarse su recuperación. Estos comportamientos derivan de la

configuración del repositorio; los resultados efectivos necesitan validación y capturas.

3. Innovaciones emergentes

La revisión identifica tres extensiones posibles del monitoreo realizado en la demo: correlacionar señales, observar tráfico con mayor detalle y asistir el diagnóstico mediante IA. Son tecnologías analizadas, no implementadas en el prototipo.

3.1. OpenTelemetry y correlación de señales

Las métricas muestran variaciones cuantitativas; los logs registran eventos y las trazas siguen una solicitud a través de distintos servicios. Relacionarlas permite investigar, por ejemplo, si un aumento de latencia coincide con un error y en qué componente aparece. Sin embargo, esa coincidencia no confirma automáticamente una causa.

OpenTelemetry ofrece mecanismos abiertos para instrumentar aplicaciones y recopilar o exportar métricas, logs y trazas [9]. Complementa, más que reemplaza, a herramientas como Prometheus y Grafana. En nuestra demo solo se recopilan métricas y comprobaciones HTTP: incorporar trazas o centralizar logs exigiría nuevos componentes.

3.2. eBPF, observabilidad a nivel de kernel y Hubble

eBPF permite ejecutar programas controlados en puntos del kernel Linux y obtener datos de eventos y comunicaciones sin modificar necesariamente las aplicaciones. Es útil cuando las métricas indican un problema, pero no revelan qué tráfico o proceso está involucrado [10].

Cilium aprovecha eBPF para conectividad y seguridad, mientras que Hubble aporta visibilidad de flujos entre servicios en Kubernetes [10]. Su adopción demanda compatibilidad, permisos y recursos. Nuestra demo no utiliza estas herramientas: blackbox_exporter solo verifica la accesibilidad HTTP, no inspecciona las comunicaciones internas.

3.3. LLM y agentes de IA aplicados a la operación de observabilidad

Los modelos de lenguaje (LLM) pueden ayudar al operador a interpretar alertas, agrupar evidencia y proponer verificaciones. Investigaciones como RCACopilot [11] y

AIOpsLab [12], así como herramientas abiertas como HolmesGPT [13], exploran este apoyo al diagnóstico. No obstante, una explicación generada por IA no constituye una causa comprobada.

El caso de la demo permite ilustrarlo: si Nginx deja de responder, Prometheus registra probe_success = 0 y genera ServicioCaido tras 30 segundos de falla sostenida. El sistema actual notifica el incidente mediante Alertmanager, MQTT y, con internet, ntfy. Como extensión teórica, un LLM podría sugerir comprobar la URL, el contenedor, los logs y la conectividad; el operador validaría la evidencia antes de actuar.

Un asistente de este tipo debería operar con permisos mínimos, proteger datos y credenciales, registrar sus recomendaciones y exigir aprobación humana antes de modificar la infraestructura. El monitoreo convencional debe seguir funcionando si la IA falla. El grupo no implementó esta integración: la presenta únicamente como propuesta futura.

Figura 2. Flujo propuesto: alerta → asistente LLM → validación humana → acción registrada. Diagrama pendiente de incorporar.

4. Oportunidades y desafíos

Disponer de más información no garantiza diagnosticar mejor. La evolución del monitoreo debe equilibrar utilidad, costo y seguridad, especialmente fuera de entornos de demostración [2].

4.1. Escalabilidad y calidad de los datos

En Prometheus, agregar etiquetas facilita distinguir servicios, pero utilizar valores casi únicos puede multiplicar las series temporales, elevar la cardinalidad y aumentar el consumo de recursos [6]. También deben definirse frecuencias de muestreo, retención y disponibilidad del propio monitoreo.

La demo recolecta cada cinco segundos y retiene datos durante dos días: es suficiente para ilustrar tendencias breves, pero no demuestra escalabilidad. Tampoco correlaciona automáticamente métricas, logs y trazas, lo que limita la investigación de causas complejas [2], [9].

4.2. Seguridad de la infraestructura de observabilidad

Las plataformas de observabilidad exponen datos sobre servicios y topologías. En producción requieren autenticación, permisos mínimos, segmentación, cifrado cuando corresponda y protección de endpoints. SNMPv3 muestra que el canal de gestión también necesita seguridad [3].

La demo prioriza facilidad de uso: Grafana emplea credenciales de ejemplo, se publican puertos locales y MQTT se configura de forma simplificada; ntfy requiere internet. Antes de un despliegue real sería necesario restringir accesos, proteger credenciales y asegurar que la supervisión local siga operativa si falla la notificación externa.

4.3. Líneas futuras de investigación y aplicación

Como mejora inmediata, conviene repetir la caída y recuperación de Nginx y medir el tiempo hasta la alerta, la llegada de notificaciones y los posibles falsos positivos. Estos ensayos aportarían evidencia verificable, sin atribuir a la demo resultados aún no medidos.

Más adelante podrían incorporarse logs, OpenTelemetry, Hubble o asistencia LLM con aprobación humana [9]–[13]. Su valor debería medirse frente al esfuerzo y los riesgos añadidos. La prioridad es construir observabilidad gradual, segura y explicable, no incorporar tecnologías por acumulación.

5. Conclusiones

[Conclusiones pendientes — Patricio: síntesis, límites y proyección (½ página).]

Referencias bibliográficas

[1] J. F. Kurose y K. W. Ross, Redes de computadoras: un enfoque descendente, 7.ª ed., Pearson, 2017, §5.7.

[2] D. G. Dutt, Cloud Native Data Center Networking, O'Reilly, 2019, cap. 11.

[3] U. Blumenthal y B. Wijnen, «User-based Security Model for SNMPv3», RFC 3414, 2002. [En línea].

[4] R. Gerhards, «The Syslog Protocol», RFC 5424, 2009. [En línea].

[5] B. Claise et al., «Specification of the IPFIX Protocol», RFC 7011, 2013. [En línea].

[6] Prometheus Authors, «Overview», documentación oficial. [En línea].

[7] Grafana Labs, «Dashboards», documentación oficial. [En línea].

[8] OpenConfig, «gNMI Specification», especificación técnica. [En línea].

[9] OpenTelemetry, «Signals», documentación oficial. [En línea].

[10] Cilium, «Introduction to Cilium & Hubble», documentación oficial. [En línea].

[11] Y. Chen et al., «Automatic Root Cause Analysis via Large Language Models for Cloud Incidents», EuroSys, 2024. [En línea].

[12] Y. Chen et al., «AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds», MLSys, 2025. [En línea].

[13] HolmesGPT, «HolmesGPT: Open-source SRE Agent», repositorio oficial. [En línea].